
Frozen Memory Is Not Enough: Rethinking External Memory as Extraction

Anonymous Author(s)

Affiliation

Address

email

Abstract

Methods for improving knowledge use in large language models typically fall into two regimes. Non-parametric retrieval offers flexible access to external knowledge, but adds retrieval latency, context overhead, and only shallow integration with the backbone. Parametric adaptation is efficient at inference time, but entangles knowledge with model weights and can be hard to update, audit, or transfer. Engram-style hashed memory occupies a middle regime: it stores learned information in an external, addressable table, yet consumes that table through a small learned reader. This raises a basic question: when such a memory is moved across backbones, what matters more, the frozen memory itself or the target-side reader? We study this question through cross-model frozen-memory extraction, where a memory trained on a source model is frozen and attached to a different target model while only a lightweight reader is trained. Across architectures, tokenizers, and model scales, frozen memory improves every source–target pair in a 3×3 transfer matrix, with up to 15.7% relative perplexity reduction. Ablations show that learned memory content and correct addressing both matter, but the transferred table becomes useful only through a reader aligned to the target model. On downstream tasks, a target-side dual-layer 4-branch reader on OpenQA nearly closes the gap between same-model and cross-model transfer, reaching a 38.78 average score, a state-of-the-art result. These results position Engram as a hybrid external memory and turn portability into an evaluation problem: useful memory is not merely stored, but must be extractable by the target backbone. Code and reproducible setups are available at https://anonymous.4open.science/r/Engram_Extractor/README.md.

1 Introduction

Language models can use knowledge in two familiar ways. *Non-parametric* methods, such as retrieval-augmented generation (RAG) [16] and nearest-neighbor retrieval [13], keep knowledge outside the backbone and make updates relatively direct, but they pay for that flexibility with retrieval latency, longer contexts, and loose integration between retrieved evidence and generation. *Parametric* methods, such as continued pretraining, fine-tuning, or learned memory layers tied to a single model [5], avoid retrieval at inference time, but they entangle knowledge with model weights. This makes updates, audits, and transfer across backbones difficult.

Engram-style hashed memory [6] occupies an interesting middle ground. It stores learned information in an explicit, addressable external table, yet consumes that table through a small neural interface rather than by inserting raw documents into the context. This hybrid design is attractive because it appears to combine the modularity of external memory with the efficiency of learned representations. But it also exposes a sharper question: **is the memory itself reusable, or is it merely a co-adapted extension of the model that trained it?**

37 The distinction matters: a memory table that is not portable is just another form of model-specific
38 parameterization, not a reusable knowledge substrate. Thus, the portability requires an operational
39 test: *remove the source backbone and ask whether another model can still extract useful signal from*
40 *the frozen table.*

41 We study this test through *cross-model frozen-memory reuse*. Given a memory table trained with
42 a source model A , we freeze the table, attach it to a different target model B , and train only a
43 lightweight target-side reader. If the frozen memory still improves the target after the original host is
44 removed, then it contains usable information beyond source-specific co-adaptation. If it fails, then
45 the apparent memory benefit was likely tied to the original backbone. Figure 1 illustrates this life
46 cycle: source-side memory training, memory export and freezing, and target-side extraction through
47 a new reader.

48 To make this test precise, we separate three roles: *addressing* decides which entries are read, *memory*
49 denotes the stored vectors, and the *reader* maps them into the model. We standardize addressing,
50 freeze the memory, and adapt only the reader. This turns portability into a measurable question: **can**
51 **a target model learn to read a fixed memory artifact?**

52 Our method instantiates this decomposition directly. We use a tokenizer-agnostic canonicalization
53 pipeline to keep memory addresses stable across tokenizers, freeze the exported source memory, and
54 train only a small target-side reader that projects memory values into the target residual stream. The
55 reader is not an incidental adapter. Because the frozen table was shaped through gradients from a
56 different host model, the reader is the main bottleneck that determines whether the stored signal can
57 be extracted at all. We therefore also study stronger multi-branch and multi-layer readers to test
58 whether improved extraction, rather than increased storage, explains the largest gains. Our scope
59 is deliberately focused: we do not claim universal portability across all memory architectures, but
60 ask whether lightweight reader adaptation is sufficient for Engram-style memory across diverse
61 backbones.

62 Our central claim is: **for portable hashed external memory, memory presence is not enough; the**
63 **target-side reader is the key extraction bottleneck.** Memory content still matters—transferred
64 memory consistently outperforms random memory and permuted-key controls—but the amount of
65 usable signal depends on how well the target model can align and integrate what it retrieves. This
66 view explains why imperfect memory can still help with a capable reader, and why reader design can
67 matter as much as retraining or enlarging the stored table in this regime.

68 We support this claim with a compact evidence chain. First, a 3×3 source–target transfer matrix
69 spanning Pythia, QWen, and TinyLlama shows that frozen memory improves all model pairs, with
70 relative perplexity reductions of up to 15.7%. Second, ablations separate memory content from
71 generic module capacity: transferred memory outperforms random memory, permuted keys, and
72 training target memory from scratch under the same target-side adaptation budget, showing that an
73 existing source memory can be reused efficiently once exported. Third, on downstream tasks, a
74 unified OpenQA study on Mistral-7B shows that upgrading the target-side reader to a dual-layer,
75 multi-branch architecture nearly eliminates the gap between same-model and cross-model reuse
76 (38.53 vs. 38.50 average), with the best configuration reaching 38.78 and selective gains on other
77 knowledge-intensive benchmarks.

78 In summary, our contributions are:

- 79 1. We formulate cross-model frozen-memory reuse as an evaluation problem for external
80 memory, separating portable memory from backbone-specific co-adaptation and measuring
81 whether the target model can extract usable signal from a frozen artifact.
- 82 2. We instantiate this problem with a transfer protocol for Engram-style memory that standard-
83 izes addressing across tokenizers and uses lightweight target-side readers, including stronger
84 multi-branch, multi-layer variants.
- 85 3. We provide an evidence chain showing that reader design is first-order: consistent gains
86 across a 3×3 transfer matrix, ablations isolating memory content and addressing integrity,
87 and OpenQA results where improved readers dominate cross-model performance.

88 Taken together, these results support an extraction-centric view of external memory. For Engram-style
89 hashed memory, the key question is not only how to store information, but whether that information
90 remains usable once the original backbone is removed.

2 Background and Related Work

2.1 Engram Architecture

Engram [6] augments a Transformer-MoE backbone with a conditional memory that retrieves static N-gram embeddings via deterministic hashing and injects them through a learned gate. For our purposes, the important feature is the split between *addressing* memory and *consuming* memory: lookup depends on the input text, while the backbone only sees the retrieved vector through a small learned interface.

A surjective vocabulary projection $\mathcal{P} : V \rightarrow V'$ maps raw token IDs to canonical IDs via NFKC normalization, lowercasing, and accent stripping, reducing the effective vocabulary by $\sim 23\%$ [6]. For each N-gram order $n \in [2, N]$ and hash head $k \in [1, K]$, a deterministic multiplicative-XOR hash $\varphi_{n,k}$ maps the canonical N-gram to an index in an embedding table $\mathbf{E}_{n,k}$ of size $M_{n,k}$. The retrieved embeddings are concatenated across all orders and heads:

$$\mathbf{e}_t \triangleq \bigoplus_{n=2}^N \bigoplus_{k=1}^K \mathbf{E}_{n,k}[\varphi_{n,k}(g_{t,n})], \quad (1)$$

where $g_{t,n} = (x'_{t-n+1}, \dots, x'_t)$ is the canonical N-gram context. Crucially, the lookup index depends only on the canonicalized input sequence, not on the model’s hidden states.

A scalar gate $\alpha_t \in (0, 1)$ mediates how much of the retrieved memory enters the hidden state. In original Engram, the gate score is passed through a signed square-root nonlinearity before the sigmoid:

$$s_t = \frac{\text{RMSNorm}(\mathbf{h}_t)^\top \text{RMSNorm}(\mathbf{k}_t)}{\sqrt{d}}, \quad \alpha_t = \sigma\left(\text{sign}(s_t)\sqrt{|s_t|}\right), \quad (2)$$

$$\mathbf{k}_t = \mathbf{W}_K \mathbf{e}_t, \quad \mathbf{v}_t = \mathbf{W}_V \mathbf{e}_t,$$

where $\mathbf{h}_t$ is the backbone hidden state and $\mathbf{W}_K, \mathbf{W}_V$ are learnable projection matrices. Our transfer-oriented reader in later sections uses a simpler sigmoid gate with an optional learned bias; Table 7 makes that change explicit. This separation between retrieval and consumption is what makes frozen-memory extraction experimentally separable: the table already behaves like a standalone artifact, while the projections and gate define a small backbone-specific interface.

2.2 Related Memory Mechanisms

MoE [21, 8, 7] expands capacity through conditional computation, but the useful signal remains entangled with the backbone that hosts it. Other memory-augmented models also miss the extraction question for one of two reasons: they either retrieve over large dynamic memory banks at inference time [9, 25], or they store useful signal in model-specific hidden spaces, as in KNN-LM [13].

Retrieval-augmented generation and RETRO-style systems [16, 4, 12] are portable at the document level, but the reusable object is a document store rather than a trained memory representation. Larger learned memory layers [15, 2, 1, 5] improve capacity, but the stored representation is still usually tied to the consuming model. Engram occupies the narrower regime relevant here: deterministic hashing yields a stable address space and constant-time lookup, which makes a portable memory artifact at least well-defined. A broader qualitative comparison appears in Appendix Table 6.

2.3 Cross-Model Alignment

Portability still requires alignment. Embedding-space work suggests that independently trained models often admit simple linear maps between related representations [18, 14, 11, 23], and PEFT transfer methods such as Cross-LoRA, LoRA-X, and Trans-LoRA [26, 28, 24] show that compact interfaces can sometimes move across models. In our setting, however, tokenizer mismatch appears one step earlier than hidden-space alignment: source and target must first agree on which memory entry to read. That is why the method begins with canonicalization of the address space and only then fits a lightweight reader.

3 Methods

Our method recasts Engram as a transferable artifact by breaking the co-adaptation between storage and consumption. We freeze the source memory and fit only a small target-side reader. This requires a shared address space, the frozen memory, and a lightweight reader. Relative to native Engram, we make three changes: we freeze the exported source memory, replace model-specific token-ID lookup with tokenizer-agnostic canonicalization, and move memory consumption into a lightweight target-side reader attached by residual injection. Figure 1 gives the setup.

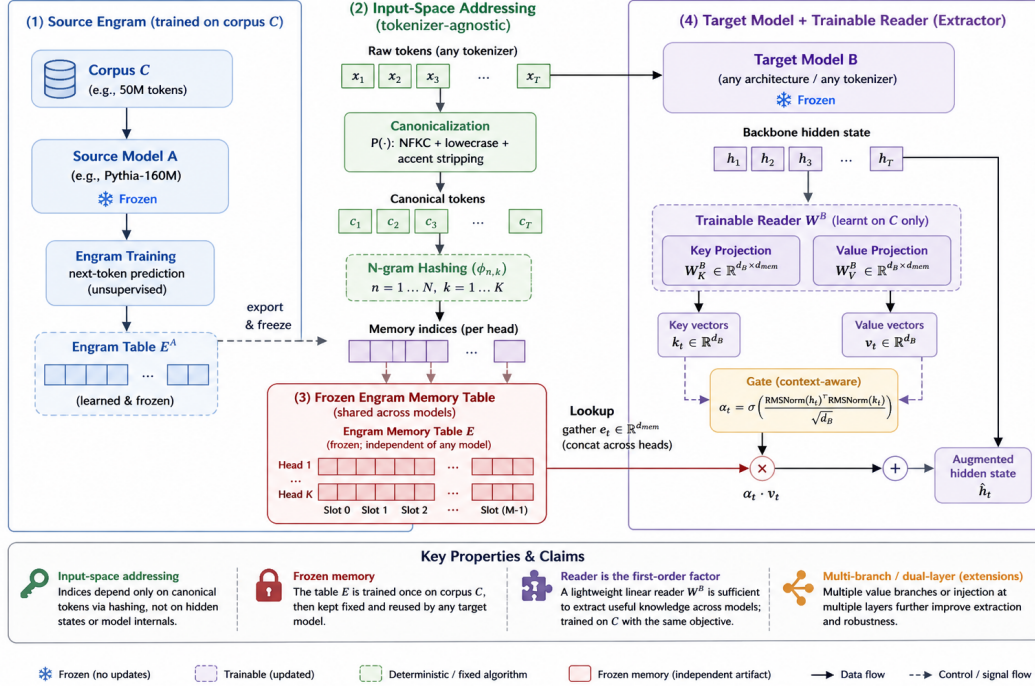


Figure 1: Cross-model frozen-memory extraction. A memory table trained with source model A is frozen and attached to target model B . Canonicalization keeps the address space fixed across tokenizers, and only the target-side reader is trained. This setup operationalizes whether external memory remains reusable outside its source backbone.

3.1 Tokenizer-Agnostic Addressing

The first obstacle to transfer is address mismatch. Native Engram hashes model-specific token IDs inside the host model, which is sufficient when storage and consumption happen under one tokenizer but does not define a stable cross-model address space. If source and target tokenizers segment the same text differently, then identical surface content may be routed to different memory entries. To avoid this failure mode, we build the lookup space on canonicalized decoded text rather than raw token IDs. Let $\mathcal{T}_B$ be the tokenizer of target model B , and let $s_{1:t} = \text{decode}(\mathcal{T}_B(x_{1:t}))$ denote the decoded target prefix up to position t . We normalize that prefix with a shared canonicalization function $\mathcal{P}$ and extract the completed word sequence

$$(w_1, \dots, w_{m(t)}) = \text{words}(\mathcal{P}(s_{1:t})), \quad (3)$$

where $\text{words}(\cdot)$ returns the canonicalized word-boundary units implied by the current decoded prefix. In practice, $\mathcal{P}$ applies NFKC normalization, lowercasing, accent stripping, and whitespace cleanup. Memory lookup is then performed over canonical word-boundary n -grams rather than model-specific token IDs:

$$\mathbf{e}_t = \bigoplus_{n=2}^N \bigoplus_{j=1}^K \mathbf{E}_{n,j} [\varphi_{n,j}(w_{m(t)-n+1}, \dots, w_{m(t)})]. \quad (4)$$

At subword positions inside the same decoded word, the current token reuses the same right-edge word context until a new word boundary appears. The retrieved vector $\mathbf{e}_t$ is therefore determined

154 by canonicalized text and the frozen memory artifact, not by target-specific token IDs. Two target
 155 models with different tokenizers can still read the same transferred entry as long as they share the
 156 same canonicalization and hash specification.

Algorithm 1: Cross-Model Frozen-Memory Extraction

Input: Source memory $\mathcal{E}_A$, target model B , target corpus $\mathcal{D}$, selected injection layers $\mathcal{L}$, branch count R , canonicalization $\mathcal{P}$
Output: Target model B augmented with frozen transferred memory

```

// // Phase 1 artifact is already trained; prepare transfer
1 Extract Engram tables  $\{\mathbf{E}_{n,j}\}$  and hash specification from  $\mathcal{E}_A$ ;
2 Freeze all transferred tables  $\{\mathbf{E}_{n,j}\}$  [portable memory artifact];
// // Initialize target-side reader and hooks
3 foreach  $\ell \in \mathcal{L}$  do
4   Initialize reader parameters  $\mathcal{W}^{(B,\ell)}$ ;
5   Set  $\mathcal{W}^{(B,\ell)} \leftarrow \{\mathbf{W}_{K,\ell,r}^{(B)}, \beta_{\ell,r}\}_{r=1}^R, \mathbf{W}_{V,\ell}^{(B)}$ ;
6   if  $R = 1$  then
7     Set  $\beta_{\ell,1} \leftarrow 0$ ;
8   Register a forward hook at layer  $\ell$  that computes  $\mathbf{o}_{t,\ell}$  from the current  $\mathbf{h}_{t,\ell}$  and looked-up  $\mathbf{e}_t$ , then
    injects  $\mathbf{h}_{t,\ell} \leftarrow \mathbf{h}_{t,\ell} + \mathbf{o}_{t,\ell}$ ;

// // Phase 2: Reader-only adaptation
9 for  $x_{1:T} \in \mathcal{D}$  do
10   Canonicalize each decoded prefix into shared word-boundary  $n$ -grams with  $\mathcal{P}$ ;
11   Look up frozen memory vectors  $\mathbf{e}_{1:T}$  from  $\{\mathbf{E}_{n,j}\}$ ;
12   Run the frozen backbone on  $x_{1:T}$  with the registered hooks active;
13   Update only reader parameters  $\{\mathcal{W}^{(B,\ell)}\}_{\ell \in \mathcal{L}}$  using the LM loss [backbone and memory frozen];

```

157 3.2 Target-Side Reader

158 Once the address space is shared, the remaining problem is extraction: how should the target model
 159 consume a memory vector whose geometry was shaped by a different backbone? We use a shared-
 160 value reader family parameterized by branch count R , where larger R increases target-side alignment
 161 capacity without increasing memory size. For branch $r \in \{1, \dots, R\}$ at layer ℓ :

$$162 \quad \mathbf{k}_{t,\ell}^{(r)} = \mathbf{W}_{K,\ell,r}^{(B)} \mathbf{e}_t, \quad \mathbf{v}_{t,\ell} = \mathbf{W}_{V,\ell}^{(B)} \mathbf{e}_t, \quad (5)$$

$$163 \quad \alpha_{t,\ell}^{(r)} = \sigma \left(\frac{\text{RMSNorm}(\mathbf{h}_{t,\ell})^\top \text{RMSNorm}(\mathbf{k}_{t,\ell}^{(r)})}{\sqrt{d_B}} + \beta_{\ell,r} \right), \quad (6)$$

$$164 \quad \mathbf{o}_{t,\ell} = \frac{1}{R} \sum_{r=1}^R \alpha_{t,\ell}^{(r)} \cdot \mathbf{v}_{t,\ell}. \quad (7)$$

164 The resulting reader output is injected through a residual connection:

$$165 \quad \mathbf{h}_{t,\ell} \leftarrow \mathbf{h}_{t,\ell} + \mathbf{o}_{t,\ell}. \quad (8)$$

165 The shared value projection forces all branches to operate on the same retrieved content and differ
 166 only in *how* they gate and align it. The added capacity is therefore an extraction mechanism rather
 167 than extra storage. The main 3×3 transfer matrix uses $R = 1$; the stronger OpenQA reader uses
 168 $R = 4$.

169 **Multi-layer injection.** The same reader family can be instantiated at one or more target layers $\mathcal{L}$.
 170 Single-layer injection is sufficient for the cross-architecture transfer matrix, whereas the best OpenQA
 171 configuration uses dual-layer injection at layers 2 and 10 with $R = 4$ branches.

172 3.3 Training Regime

173 The training objective is the standard next-token language modeling loss on the target corpus. During
 174 target-side adaptation, both the exported source memory tables $\{\mathbf{E}_{n,j}\}$ and the target backbone

parameters θ_B remain frozen; only the reader parameters $\{\mathcal{W}^{(B,\ell)}\}_{\ell \in \mathcal{L}}$ are updated. Storage is fixed and only extraction is learned. Algorithm 1 summarizes the procedure concisely. The key operations are to freeze the exported source memory, canonicalize target inputs into the shared address space, and update only the target-side reader.

4 Experiments and Results

Implementation details are in Appendix C. All experiments follow the same protocol: a source memory is trained (Phase 1), then frozen and attached to a target model where only a reader is trained (Phase 2). Intrinsic evaluations use a minimal reader; OpenQA uses a stronger multi-layer, multi-branch reader.

4.1 RQ1: Does frozen memory transfer across backbones, tokenizers, and scales?

Our first question is whether the exported memory remains useful once detached from the source backbone. We start with a 3×3 source–target matrix spanning three source memories (Pythia-160M, QWen3.5-0.8B, QWen3.5-9B) and three target models (Pythia-410M, QWen3.5-4B, TinyLlama-1.1B), all trained with the same 20M-token target-side protocol.

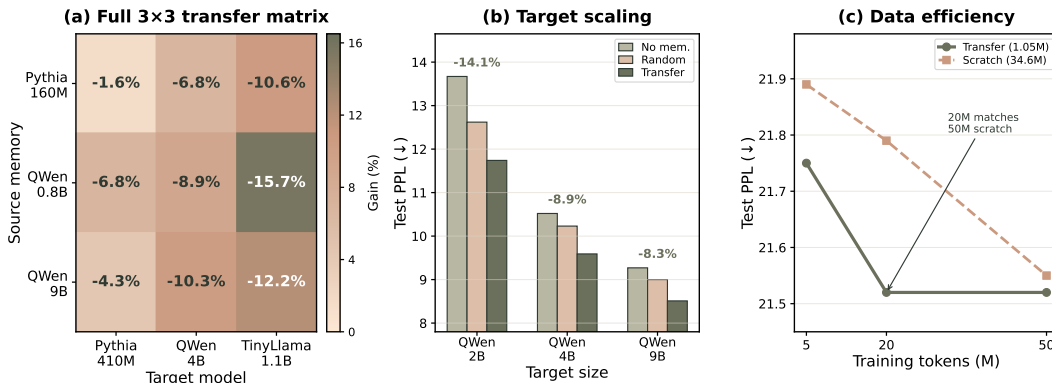


Figure 2: Main-text visual summary for RQ1 and RQ2. The left panel shows the full 3×3 transfer matrix, where all source–target pairs obtain positive perplexity reductions over the no-memory baseline. The middle panel shows that the same transferred QWen-0.8B memory continues to help as the target scales from 2B to 9B. The right panel shows the target-side data-efficiency curve: a 1.05M-parameter transferred reader reaches essentially the same regime by 20M target tokens that a 34.6M-parameter from-scratch memory only approaches near 50M tokens.

The full matrix gives a positive answer in every one of the nine cells. Transferred memory improves over the no-memory baseline in all source–target pairs, with relative gains from 1.6% to 15.7%, and it consistently beats random memory. The strongest gain appears on QWen-0.8B $\rightarrow$ TinyLlama, showing that large improvements do not require source and target to share architecture or tokenizer. Figure 2 summarizes this result together with the scaling trend: all nine cells in the main matrix are positive, and the same transferred QWen-0.8B memory continues to help as the target grows from 2B to 9B, although the relative gain narrows as the target becomes stronger. This is also not only an intrinsic-perplexity effect: Table 4 later shows selective downstream gains for QWen targets from 0.8B to 9B, and Appendix Table 14 shows that the same target family retains that pattern under two different source-corpus families.

One subtle pattern is that source scaling helps, but not monotonically for every target. QWen-0.8B improves over Pythia-160M on all three targets, yet QWen-9B does not always beat QWen-0.8B on the most dissimilar target (Pythia-410M), suggesting that transfer depends not only on what the source memory stores, but also on what the target reader can successfully extract. This underscores the importance of the reader’s extraction capability, as appendix results confirm that transfer is effective in peer-to-peer settings (§D.4) and even between backbones with low representational similarity (§D.6).

Takeaway. RQ1 is answered positively: frozen memory transfer remains effective across backbone families, tokenizer boundaries, and target scales, with gains in all nine main source–target pairs; the same QWen target family also retains selective downstream gains beyond perplexity, and successful transfer ultimately depends on whether the target reader can extract the stored structure.

4.2 RQ2: What drives the gains, and how data-efficient is transfer?

The next question is whether the gains come from meaningful transferred memory or from adding another trainable module. We answer this with a compact ablation on Pythia-160M $\rightarrow$ Pythia-410M.

Table 1: Ablation study for Pythia-160M $\rightarrow$ Pythia-410M. Changes in Δ PPL are measured relative to the transferred condition; red values indicate higher (worse) perplexity. All comparisons use paired t -tests on per-batch perplexities.

Condition	Test PPL $\downarrow$	Δ PPL	p -value	Params
Transferred (full)	21.54 $\pm$ 0.05	—	—	1.05M
<i>Interface Simplifications</i>				
No gate	21.55 $\pm$ 0.01	+0.01	0.23	524K
Affine stitch	21.56 $\pm$ 0.00	+0.02	< 0.001	525K
<i>Content and Training Controls</i>				
Permuted keys	21.78 $\pm$ 0.01	+0.23	< 0.001	1.05M
Train from scratch	21.79 $\pm$ 0.01	+0.25	< 0.001	34.6M
Random memory	21.85 $\pm$ 0.00	+0.31	< 0.001	1.05M
Baseline (no memory)	21.90	+0.36	—	—
<i>Degenerate Capacity Control</i>				
FFN only (no memory)	40.02 $\pm$ 4.21	+18.48	< 0.001	34K

Table 1 supports three conclusions. The gain is not a generic reader-capacity effect: random memory and permuted keys are both substantially worse than transferred memory, the FFN-only control collapses, and as shown in the appendix (Table 9), the method outperforms parameter-matched PEFT baselines like LoRA. Pretraining the memory also matters: once the source table has been exported, a transferred setup with only 1.05M trainable target-side parameters outperforms training a new target memory from scratch even though the latter updates 34.6M parameters during Phase 2. This should not be read as a claim that transfer uses less total pretraining compute; rather, the result shows that a reusable memory artifact amortizes source-side training and makes subsequent target adaptation data-efficient. By contrast, the exact interface form is less critical in this in-domain setting, since removing the gate or replacing it with an affine stitch changes performance only marginally.

The right panel of Figure 2 points in the same direction. The transferred reader reaches its best target-adaptation regime by 20M target-side tokens and already performs well at 5M, whereas the from-scratch memory only approaches the same level near 50M despite updating far more parameters. Appendix Table 13 provides the exact values.

The same conclusion survives task-level evaluation. In Table 2, the best transferred Mistral OpenQA system reaches 38.50–38.53 average, while hidden-size-compatible frozen-reader reuse remains nearly as strong at 38.30–38.31. These frozen-reader rows reuse the source-side reader directly at evaluation time, with no target-side fitting, and are possible here because the relevant 7B source and target backbones expose compatible residual widths. They therefore test whether the exported artifact is fragile, not whether target-side reader training is unnecessary. By contrast, memory-only collapses to 7.02 average, permuted keys fall to 33.62, and the FFN-only controls fail to match the transferred system, especially at the larger $R=4$ capacity. These downstream controls reinforce that the gain comes from learned memory content, correct addressing, and a reader that can align that content to the target model rather than from adding a generic target-side module.

Takeaway. RQ2 is answered by both intrinsic and task-level controls: the gain comes from meaningful transferred memory and correct addressing rather than generic reader capacity, and transfer is data-efficient because a small target-side reader reaches its best regime with far fewer trainable parameters and fewer Phase 2 tokens than training a new memory from scratch.

4.3 RQ3: How much does reader design matter?

Our main claim is that extraction quality is a first-order bottleneck. We test that claim directly on OpenQA with a stronger reader family. The main system here is the shared-value multi-branch reader with injection at target layers $\{2, 10\}$. To keep the notation unambiguous, we write the injection set explicitly as “inj. $\{2, 10\}$ ” rather than “2L”, use R for branch count, and spell out the source model as “LLaMA src.” or “Mistral src.” instead of a single letter.

Table 2 shows that the best reader with inj. $\{2, 10\}$ and $R=4$ reaches 38.78 average, while same-model and cross-model transfer are nearly tied at 38.53 and 38.50. This is the clearest evidence that source-backbone identity is not the dominant factor once the target has a reader capable of extracting the memory. The control runs sharpen that conclusion: frozen-source-reader variants already reach 38.30–38.31 without any target-side reader training, so the transferred memory artifact is not fragile, yet they rely on residual-width compatibility and do not remove the need for target reader alignment in the general case. FFN-only and permuted-key controls explain why: the reader must do meaningful routing, but that routing still depends on access to correctly addressed structured memory.

Table 2: OpenQA accuracy (%) on Mistral-7B-v0.3 for transfer verification and controls. Deltas are measured against the baseline, and the percentage in the Average column denotes relative improvement over the baseline average. Budget = $P1/P2$, where $P1$ is the source-memory training budget and $P2$ is the target-side reader fitting budget (millions of tokens). The shorthand $\{a, b\}$ -R r means injection at target layers $\{a, b\}$ with r branches; “frz” means the source-side reader is frozen at evaluation time with no target-side fitting, possible here because the source and target residual widths are compatible.

Variant	Budget	NQ	WebQA	TriviaQA	TruthQA	Hotpot	Average
Base	–	20.63	29.28	57.65	32.09	20.96	32.12
<i>Transfer Verification</i>							
Mistral $\{2, 10\}$ -R4	10/20	30.17 9.54	33.95 4.67	69.99 12.34	30.84 -1.25	27.72 6.76	38.53 (+20.0%)
LLaMA $\{2, 10\}$ -R4	10/20	30.34 9.71	33.67 4.39	69.94 12.29	30.94 -1.15	27.62 6.66	38.50(+19.9%)
Frozen Mistral	10/0	30.46 9.83	30.26 0.98	70.81 13.16	32.24 0.15	27.72 6.76	38.30(+19.2%)
Frozen LLaMA	10/0	30.63 10.00	30.27 0.99	70.79 13.14	32.21 0.12	27.62 6.66	38.31(+19.3%)
<i>Controls and Ablations</i>							
Mem-only Mistral	10/0	0.19 -20.44	0.28 -29.00	1.35 -56.30	33.03 0.94	0.23 -20.73	7.02(-78.1%)
Mem-only LLaMA	10/0	0.00 -20.63	0.00 -29.28	0.00 -57.65	–	0.00 -20.96	–
Permuted	10/0	20.64 0.01	28.85 -0.43	63.07 5.42	32.39 0.30	23.13 2.17	33.62(+4.7%)
FFN R4	10/20	19.59 -1.04	27.32 -1.96	58.10 0.45	32.08 -0.01	17.55 -3.41	30.93(-3.7%)
<i>Token Scaling (LLaMA2-7B source, $\{2, 10\}$-R4)</i>							
Engram	10/10	30.17 9.54	29.86 0.58	70.41 12.76	31.19 -0.90	27.64 6.68	37.85(+17.8%)
Engram	15/15	30.22 9.59	32.67 3.39	70.08 12.43	30.62 -1.47	27.75 6.79	38.27(+19.1%)
Engram	20/20	30.29 9.66	33.86 4.58	69.93 12.28	30.67 -1.42	27.64 6.68	38.48(+19.8%)
Engram	25/25	30.52 9.89	33.09 3.81	69.95 12.30	30.92 -1.17	27.65 6.69	38.43(+19.6%)
Engram	30/30	30.59 9.96	33.89 4.61	70.35 12.70	31.14 -0.95	27.92 6.96	38.78 (+20.7%)

An appendix reader-variant table further separates *placement* from *branching*: using two well-chosen injection layers already helps substantially, and adding four branches improves extraction further. The gain therefore does not come from “more layers” alone.

Table 3 makes that pattern explicit. A single-branch reader injected only at layer 10 ($R=1$) reaches only 33.98–34.23 average. Moving to inj. $\{2, 10\}$ while keeping $R=1$ raises performance to 37.49, and increasing the branch count to $R=4$ further raises it to 38.50. The gain therefore does not come from “more layers” alone; additional branches improve the reader’s gating and alignment flexibility on top of better placement.

Takeaway. RQ3 is answered clearly: reader design is a first-order factor. Better layer placement already improves extraction substantially, and adding more branches improves it further, nearly eliminating the same-model versus cross-model gap on OpenQA once the reader is strong enough.

4.4 RQ4: When does transfer help beyond perplexity, and where are the boundaries?

The final question is where transferred memory remains useful beyond WikiText perplexity. Table 4 shows the clearest positive regime: under corpus-matched Phase 2 adaptation, Qwen targets gain most consistently on RTE and SciQ, smaller and mid-scale targets also gain on BoolQ, and TruthfulQA is consistently slightly negative. The main picture is therefore selective rather than universal: transfer helps most on aligned factual and reasoning-heavy tasks.

Table 3: OpenQA accuracy (%) on Mistral-7B-v0.3 across reader and baseline variants. Deltas are measured against the baseline, and the Average column reports relative improvement over the baseline average. Budget notion, shorthand and “Frozen” setting are the same as in Table 2.

Variant	Budget	NQ	WebQA	TriviaQA	TruthQA	Hotpot	Average
Base	–	20.63	29.28	57.65	32.09	20.96	32.12
<i>Non-parametric Methods</i>							
RAG [16]	–	22.56 1.93	24.90 -4.38	54.21 -3.44	35.47 3.38	29.77 8.81	33.38(+3.9%)
kNN [13]	–	21.05 0.42	30.51 1.23	57.77 0.12	32.33 0.24	21.20 0.24	32.57(+1.4%)
<i>Parametric Methods</i>							
CPT	–	12.16 -8.47	34.06 4.78	61.21 3.56	29.18 -2.91	16.04 -4.92	30.53(-5.0%)
LoRA [10]	–	18.17 -2.46	34.50 5.22	61.60 3.95	30.91 -1.18	16.23 -4.73	32.28(+0.5%)
MLP Memory [5]	–	25.20 4.57	37.45 8.17	60.99 3.34	32.54 0.45	24.14 3.18	36.06(+12.3%)
<i>Engram Variants (sorted by Average)</i>							
LLaMA frz {10}	10/0	21.79 1.16	29.61 0.33	60.57 2.92	33.14 1.05	22.32 1.36	33.49(+4.3%)
Mistral frz {10}	10/0	21.86 1.23	29.71 0.43	60.79 3.14	32.91 0.82	22.38 1.42	33.53(+4.4%)
Mistral {10}-R1	10/20	22.13 1.50	31.64 2.36	61.06 3.41	32.76 0.67	22.29 1.33	33.98(+5.8%)
LLaMA {10}-R1	10/20	22.17 1.54	32.57 3.29	61.22 3.57	32.87 0.78	22.30 1.34	34.23(+6.6%)
LLaMA {2, 10}-R1	10/20	30.33 9.70	28.65 -0.63	70.09 12.44	31.00 -1.09	27.40 6.44	37.49(+16.7%)
LLaMA {2, 10}-R4	10/20	30.34 9.71	33.67 4.39	69.94 12.29	30.94 -1.15	27.62 6.66	38.50 (+19.9%)

Table 4: Knowledge-intensive downstream task evaluation: accuracy change ($\Delta\%$) from transferred memory vs. no-memory baseline, using a QWen3.5-9B source memory trained on FineWeb-Edu (FW-9B) and a corpus-matched Phase 2 reader. Positive gains are shown in green and negative changes in red.

Target	RTE $\uparrow$	BoolQ $\uparrow$	OpenBookQA $\uparrow$	SciQ $\uparrow$	TruthfulQA $\uparrow$	RACE $\uparrow$
QWen3.5-0.8B	+3.7 $\pm$ 4.9	-1.3 $\pm$ 3.2	+0.7 $\pm$ 0.3	+2.2 $\pm$ 1.0	-0.2 $\pm$ 0.1	-0.3 $\pm$ 0.4
QWen3.5-2B	+3.5 $\pm$ 1.9	+3.9 $\pm$ 1.4	+0.6 $\pm$ 0.4	+3.7 $\pm$ 0.6	-0.5 $\pm$ 0.3	-0.1 $\pm$ 0.1
QWen3.5-4B	+1.4 $\pm$ 0.3	+0.1 $\pm$ 0.2	+0.2 $\pm$ 0.7	+0.7 $\pm$ 0.6	-0.6 $\pm$ 0.2	+0.1 $\pm$ 0.2
QWen3.5-9B	+0.7 $\pm$ 0.3	+0.6 $\pm$ 0.2	+0.3 $\pm$ 0.3	+3.2 $\pm$ 0.7	-0.8 $\pm$ 0.3	+1.1 $\pm$ 0.2

Target baseline accuracy ranges: RTE 61–82, BoolQ 67–86, OBQA 35–45, SciQ 59–71, TruthfulQA 27–30, RACE 34–37

The appendix sharpens this picture in two ways. First, source corpus structure can matter more than raw scale; for instance, appendix results (Table 15) show a specialized 8B-token source memory decisively outperforming a generic 26B-token source on key downstream tasks, demonstrating that memory quality is not a simple function of data volume. Second, alignment between the frozen memory and the target-side adaptation regime is critical: gains that weaken under WikiText Phase 2 recover once the reader is trained on a matched HQ-DQA distribution (Table 16). TruthfulQA likely exposes the same boundary from a different angle. Unlike NQ, SciQ, or RTE, its evaluation rewards resistance to common falsehoods and calibrated non-imitation of web-distribution misconceptions; a memory trained to inject frequent factual associations can therefore help recall while slightly hurting truthfulness calibration. Outside this aligned regime, out-of-domain perplexity effects are mostly neutral rather than catastrophic.

Takeaway. RQ4 is answered selectively rather than universally: transferred memory helps beyond perplexity when the source corpus, frozen memory, and Phase 2 reader training are well aligned with factual and reasoning-heavy evaluation tasks, while truthfulness-style calibration and out-of-domain settings tend to produce neutral or weaker gains rather than consistent improvements.

5 Conclusion

We study whether a learned external memory remains useful after it is detached from the backbone that trained it and attached to a different one. For Engram-style hashed memory, the answer is yes: frozen memory improves the target across tokenizers, hidden sizes, and architectures, including every cell of the main 3×3 transfer matrix and both directions of peer transfer. The main lesson is that portability depends not only on the frozen table but also on the target-side reader: stronger readers nearly close the same-model/cross-model gap on OpenQA, and transferred memory consistently beats random memory, permuted keys, FFN-only controls, and from-scratch target memory. More broadly, cross-model frozen-memory extraction turns external-memory portability into an evaluation framework: a memory artifact should be judged by whether a new backbone can address it, align to it, and extract useful signal without retraining the stored table.

References

- [1] Ali Behrouz and Peilin Zhong. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025. URL <https://arxiv.org/abs/2501.00663>.
- [2] Vincent-Pierre Berges, Hengrui Zhao, Christopher Ré, and Michael Zhang. Memory layers at scale. *arXiv preprint arXiv:2412.09764*, 2025. URL <https://arxiv.org/abs/2412.09764>. Accepted at ICLR 2025.
- [3] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL <https://arxiv.org/abs/2304.01373>.
- [4] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, pages 2206–2240, 2022. URL <https://arxiv.org/abs/2112.04426>.
- [5] Xiang Chen, Lei Li, Ningyu Zhang, Xiaozhuan Liang, Shumin Deng, and Huajun Chen. MLP memory: Towards pretrained memory module for long-context LLMs. *arXiv preprint arXiv:2508.01832*, 2025. URL <https://arxiv.org/abs/2508.01832>.
- [6] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*, 2026. URL <https://arxiv.org/abs/2601.07372>.
- [7] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024. URL <https://arxiv.org/abs/2401.06066>.
- [8] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23 (120):1–39, 2022. URL <https://jmlr.org/papers/v23/21-0998.html>.
- [9] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014. URL <https://arxiv.org/abs/1410.5401>.
- [10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- [11] Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL <https://arxiv.org/abs/2405.07987>.
- [12] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251): 1–43, 2023. URL <https://jmlr.org/papers/v24/23-0037.html>.

- [13] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020. URL <https://arxiv.org/abs/1911.00172>.
- [14] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey E. Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. URL <https://arxiv.org/abs/1905.00414>.
- [15] Guillaume Lample, Alexandre Sablayrolles, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. URL <https://arxiv.org/abs/1907.05242>.
- [16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- [17] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017. URL <https://arxiv.org/abs/1609.07843>.
- [18] Tomas Mikolov, Quoc V. Le, and Ilya Sutskever. Exploiting similarities among languages for machine translation. *arXiv preprint arXiv:1309.4168*, 2013. URL <https://arxiv.org/abs/1309.4168>.
- [19] John X. Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M. Rush. Text embeddings reveal (almost) as much as text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12448–12460, 2023. doi: 10.18653/v1/2023.emnlp-main.765. URL <https://aclanthology.org/2023.emnlp-main.765/>.
- [20] Suvansh Sanjeev and Anton Troynikov. Embedding adapters. Technical report, Chroma Research, 2024. URL <https://research.trychroma.com/embedding-adapters>.
- [21] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017. URL <https://arxiv.org/abs/1701.06538>.
- [22] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [23] Curt Tigges, Michael Hodel, Thomas Baumgartner, Tessa Masis, Bruno Olshausen, and Elie Pavlick. Transferring linear features across LMs with model stitching. *arXiv preprint arXiv:2506.06609*, 2025. URL <https://arxiv.org/abs/2506.06609>.
- [24] Yifan Wang, Minxin Zhang, Haonan Wang, and Fei Wu. Trans-LoRA: Towards data-free transferable parameter efficient finetuning. *arXiv preprint arXiv:2405.17258*, 2024. URL <https://arxiv.org/abs/2405.17258>.
- [25] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. URL <https://arxiv.org/abs/1410.3916>.
- [26] Wenhan Xia, Chengwei Qin, and Elad Hazan. Cross-LoRA: Data-free cross-model LoRA transfer via SVD alignment. *arXiv preprint arXiv:2508.05232*, 2025. URL <https://arxiv.org/abs/2508.05232>.
- [27] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2306.01708>.

- [28] Minxin Zhang, Feihan Li, Haonan Wang, and Fei Wu. LoRA-X: Training-free LoRA transfer via subspace similarity. *arXiv preprint arXiv:2501.16559*, 2025. URL <https://arxiv.org/abs/2501.16559>.
- [29] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024. URL <https://arxiv.org/abs/2401.02385>.

A Problem Formulation: Frozen-Memory Extraction Across Models

The central claim of this paper is that cross-model frozen-memory extraction is the operational test of whether external memory is genuinely decoupled from the backbone that consumes it. The concrete question is whether a frozen table trained with source model A can still provide useful signal to target model B after only lightweight interface alignment. This turns portable memory from a qualitative property into an evaluation framework: a memory artifact is useful only to the extent that a target-side reader can extract it under fixed addressing and frozen storage. In this section we formalize that test for Engram-style hashed memory. Table 5 summarizes the notation used throughout this paper.

Table 5: Notation used throughout this paper.

Symbol	Meaning
A, B	Source model and target model, respectively.
$\mathcal{E}_A$	Trained Engram memory table from source model A .
d_A, d_B	Hidden dimensions of models A and B .
d_{mem}	Dimensionality of the memory vector $\mathbf{e}_t$.
$\mathcal{P}$	Canonicalization function: raw token string $\rightarrow$ canonical string.
V, V'	Raw vocabulary and canonical vocabulary, respectively.
$\varphi_{n,k}$	Deterministic hash function for N-gram order n , head k .
$\mathbf{e}_t$	Concatenated memory vector at position t .
$\mathbf{h}_t$	Backbone hidden state at position t .
$\mathbf{W}_K^{(B)}, \mathbf{W}_V^{(B)}$	Target model B ’s reader projection matrices.
α_t	Context-aware scalar gate value at position t .
$\mathcal{W}^{(B)}$	Full reader parameter set: $\{\mathbf{W}_K^{(B)}, \mathbf{W}_V^{(B)}, \text{gate params}\}$.

A.1 Problem Statement

Problem statement. Let model A be a source model equipped with a trained Engram memory table $\mathcal{E}_A$, and let model B be a target model with hidden dimension d_B , tokenizer $\mathcal{T}_B$, and potentially a different architecture. We ask whether the frozen memory $\mathcal{E}_A$ can provide extractable external knowledge to B through a lightweight reader $\mathcal{W}^{(B)}$, even when $d_B \neq d_A$ and $\mathcal{T}_B \neq \mathcal{T}_A$.

This is a deliberately narrow question. We restrict attention to Engram-style hashed memory, we allow lightweight target-side reader training, and we do not assume zero-data or universal transfer across all memory architectures. Under this scope, successful extraction is meaningful only if three conditions can be met simultaneously:

- Deterministic addressing:** memory lookup indices depend only on the input token sequence through a fixed canonicalization-and-hashing pipeline. They do not depend on the backbone’s internal hidden states. The *same text* can therefore map to the *same memory entries* across backbones, provided the key pipeline is standardized.
- Architectural decoupling:** the memory table $\mathcal{E}$ is consumed through learned projection matrices $\mathbf{W}_K$ and $\mathbf{W}_V$. These projections are natural reader interfaces because they already mediate between the memory space and the backbone hidden space.
- Gate-based robustness:** even if some transferred memory entries are imperfectly aligned, the sigmoid gate α_t can softly suppress their contribution. In practice, this requires the reader to learn a sufficiently negative match score and/or bias for unhelpful memory, thereby reducing interference with the hidden state.

417 The remainder of this section turns these three conditions into a concrete problem decomposition: an
 418 extraction mechanism must unify the key space, align the value space, and preserve robustness when
 419 alignment is incomplete.

420 A.2 Key-Space Unification

421 The first obstacle is address compatibility. In Engram’s original design, the hash key is computed
 422 over model-specific canonical token IDs. If model A assigns token ID 1823 to “San” and model B
 423 assigns ID 47, the resulting hash indices differ even when the underlying text is the same, so the two
 424 models would read unrelated memory entries.

425 To remove this mismatch, we define a *tokenizer-agnostic canonical key pipeline*. Let $\text{decode}(\cdot)$ denote
 426 the mapping from a tokenizer’s token sequence to its surface string. For a shared decoded text prefix,
 427 both models first normalize the same surface form before hashing:

$$\mathcal{P}(\text{decode}(\mathcal{T}_A(x_{1:t_A}^{\text{text}}))) = \mathcal{P}(\text{decode}(\mathcal{T}_B(x_{1:t_B}^{\text{text}}))), \quad (9)$$

428 where $x_{1:t_A}^{\text{text}}$ and $x_{1:t_B}^{\text{text}}$ denote source and target token prefixes that decode to the same text span, and
 429 $\mathcal{P}$ applies NFKC normalization, lowercasing, and accent stripping. In this construction, both models
 430 map the same decoded text to the same canonical form before hashing, so they can compute identical
 431 lookup indices into the shared memory table.

432 **Subword boundary misalignment.** The remaining difficulty is that tokenizers may segment the
 433 same word into different subword sequences. Three mitigations are available, in increasing order
 434 of robustness: (i) the original NFKC-based canonicalization when vocabularies overlap heavily; (ii)
 435 word-boundary N-grams computed over decoded strings; and (iii) character- or byte-level N-gram
 436 hashing. We adopt strategy (ii) as a practical middle ground: it substantially reduces tokenizer
 437 dependence without moving all the way to byte-level modeling. Our current experiments cover GPT-
 438 NeoX, LLaMA, QWen, and Phi tokenizers; languages or tokenizers without clean word boundaries
 439 may require the byte-level alternative.

440 A.3 Value-Space Alignment via Reader

441 Key agreement ensures that the target model reads the *same* memory entry as the source model;
 442 it does not ensure that the retrieved vector is meaningful in the target residual stream. Model A ’s
 443 memory vectors $\mathbf{e}_t \in \mathbb{R}^{d_{\text{mem}}}$ were shaped by gradients flowing through A ’s hidden geometry, which
 444 model B does not share.

445 Research on embedding space alignment [18, 14, 20] suggests that representational spaces of models
 446 trained on similar data are often related by an approximately linear transformation. The Platonic
 447 Representation Hypothesis [11] offers one interpretation of this phenomenon, and Chen et al. [23]
 448 show empirically that affine maps can transfer linear features across LLMs. This motivates the
 449 smallest plausible bridge between source memory and target backbone:

$$\mathbf{W}_V^{(B)} : \mathbb{R}^{d_{\text{mem}}} \rightarrow \mathbb{R}^{d_B}, \quad \mathbf{W}_K^{(B)} : \mathbb{R}^{d_{\text{mem}}} \rightarrow \mathbb{R}^{d_B}. \quad (10)$$

450 The memory table $\mathcal{E}_A$ remains *frozen*; only the reader parameters $\mathcal{W}^{(B)}$ are trained. For typical
 451 configurations ($d_{\text{mem}} = 512$, $d_B = 4096$), the reader size is

$$|\mathcal{W}^{(B)}| = 2 \cdot d_{\text{mem}} \cdot d_B + 2 \cdot d_B \approx 4.2\text{M}. \quad (11)$$

452 This is well within the LoRA-scale budget and small relative to multi-billion-parameter backbones.
 453 The linear choice is deliberate: it isolates whether cross-model reuse is already feasible under a
 454 minimal alignment assumption before introducing richer readers.

455 A.4 Gating as a Robustness Mechanism

456 Any transfer mechanism also needs a fallback when the transferred memory is unhelpful. The
 457 context-aware gate provides exactly such a mechanism. If a memory entry is poorly aligned, for
 458 example because of a hash collision or because the source geometry does not translate cleanly to the
 459 target, the key vector $\mathbf{k}_t = \mathbf{W}_K^{(B)} \mathbf{e}_t$ will be geometrically dissimilar to the backbone query $\mathbf{h}_t$. By
 460 learning a sufficiently negative match score or bias in those cases, the reader can push α_t toward zero

and suppress the memory contribution. The model can therefore tolerate a non-trivial amount of noisy or only partially aligned transferred memory without forcing that signal into the residual stream. This self-suppression is analogous to a soft mask in attention, but it operates on the memory channel itself and requires no explicit confidence model.

A.5 Task-Amortized Adaptation

The decomposition above also changes how the reader should be interpreted. Standard parameter-efficient fine-tuning (PEFT) methods, of which LoRA [10] is the most common example, usually bind a trained module to a downstream task by optimizing a task-specific loss on labeled task data. Serving N downstream tasks with the same target backbone therefore typically requires N task-specific modules.

Engram transfer follows a different training pattern. The source memory $\mathcal{E}_A$ is trained on a corpus $\mathcal{C}$ with an unsupervised next-token objective; it stores corpus-level N-gram statistics rather than task labels. The target reader $\mathcal{W}^{(B)}$ is trained with the same unsupervised objective on the same or aligned corpus family, and its role is only to project those corpus-level memory features into the target geometry. This does not eliminate the cost of Phase 1 source-memory training. Instead, it amortizes that cost: once a memory artifact exists, each new target model trains only a comparatively small reader. The reader is therefore more naturally viewed as binding to the pair $(B, \mathcal{C})$ than to a specific downstream task. We can summarize this contrast as

$$\text{LoRA: } \mathcal{W}_{\text{LoRA}} = f(B, \mathcal{T}), \quad \text{Engram: } \mathcal{W}^{(B)} = f(B, \mathcal{C}), \quad (12)$$

where $\mathcal{T}$ denotes a downstream task and $\mathcal{C}$ denotes the source corpus. This does not guarantee gains on every future task, but it does change the adaptation interface: adding a new downstream task does not require defining and optimizing a new task-specific reader objective. The same distinction explains why §F.1 can evaluate one trained reader across several unrelated benchmarks without retraining. We refer to this reuse pattern as *task-amortized adaptation*.

A.6 Open Problems

We identify five concrete sub-problems constituting a structured research agenda:

Problem 1: Universal key pipeline. Define and evaluate a tokenizer-agnostic key pipeline that is BPE-robust and backward-compatible with Engram’s existing per-token hashing. *Metric:* key-space overlap rate across {LLaMA-3, Mistral, DeepSeek-V3, GPT-4o} tokenizers as a function of N-gram order.

Problem 2: Reader data efficiency. How much fine-tuning data does the reader require to converge, and how does this depend on source/target model similarity? Our scaling experiments (§E.1) show near-saturation at $\sim 20\text{M}$ tokens with the transferred reader matching from-scratch training at 50M tokens, consistent with the linear structure of the alignment.

Problem 3: Cross-architecture transfer bounds. Our CKA analysis (§D.6) shows that transfer succeeds even with near-zero backbone similarity ($\text{CKA} < 0.014$ between Pythia and TinyLlama). Whether this extends to fundamentally different architectures (e.g., Transformer vs. Mamba/SSM) remains open.

Problem 4: Memory merging. Can Engram tables from multiple source models be merged (e.g., legal-domain from model A with medical-domain from model B)? Sparsity-based merging analogous to TIES-Merging [27] could exploit the structural sparsity of hash tables.

Problem 5: Privacy in shared ecosystems. If a shared Engram table is distributed across models, what are the membership inference and semantic inversion risks? Vec2Text attacks [19] and differential privacy mechanisms warrant evaluation.

B Supplementary Method and Comparison Details

This appendix keeps only the supplementary material that is referenced by the main paper. We group the evidence by research question so that each table stays close to the claim it supports.

Table 6 situates frozen-memory extraction relative to retrieval-based, native-memory, and model-editing approaches. The main design distinction is that our method keeps the explicit-memory benefits of Engram while making the memory artifact portable across backbones through reader-only fitting.

Table 6: Qualitative comparison of knowledge augmentation approaches. ✓ = fully supported, ◦ = partially supported, ✗ = not supported.

Property	KNN-LM	RAG	RETRO	Mem. Layers	LLM Mod.	Engram	Ours
$O(1)$ retrieval	✗	✗	✗	✗	✗	✓	✓
Parametric (trained)	✗	✗	✓	✓	✓	✓	✓
Cross-model portable	✗	✓	✗	✗	◦	✗	✓
Surgical deletion	✗	✓	✗	✗	✗	✓	✓
No context overhead	✗	✗	◦	✓	✓	✓	✓
Reader-only integration	N/A	N/A	✗	✗	✗	✗	✓

Table 7 then makes the architectural change explicit at the formula level. Relative to original Engram, our transfer-oriented version replaces native within-block integration with a target-side residual reader over a shared frozen memory, which is exactly what enables cross-model transfer in the experiments.

Aspect	Original Engram	Our transfer-oriented Engram reader
Memory indexing	Layer-specific compressed-token n -gram hashing: $r_t^{(l,n,j)} = \left(\bigoplus_{k=0}^{n-1} c(x_{t-k}) \cdot a_k^{(l)} \right) \bmod p_{l,n,j}$ where $c(\cdot)$ is the compressed tokenizer mapping and $p_{l,n,j}$ is a head-specific table size.	Shared word-level / canonicalized hashing: $r_t^{(n,j)} = (H(\text{canon_ngram}_t^{(n)}) \oplus s_j) \bmod M$ with a shared fixed table size M across heads/layers.
Memory representation	Layer-specific Engram embedding: $e_t^{(l)} = \text{Concat}_{n,j} \mathbf{E}_{l,n,j}[r_t^{(l,n,j)}]$	Shared Engram memory vector: $m_t = \text{Concat}_j \mathbf{T}_j[r_t^{(j)}]$ In our main setup, $d_{\text{mem}} = 512$.
Key / gate formula	$s_t = \frac{\langle \text{RMSNorm}(W_k e_t), \text{RMSNorm}(h_t) \rangle}{\sqrt{d}}$ $\alpha_t = \sigma \left(\text{sign}(s_t) \sqrt{ s_t } \right)$	For branch r : $k_t^{(r)} = W_k^{(r)} m_t, \quad v_t = W_v m_t$ $\alpha_t^{(r)} = \sigma \left(\frac{\langle \text{RMSNorm}(h_t), \text{RMSNorm}(k_t^{(r)}) \rangle}{\sqrt{d}} + \beta_r \right)$ where β_r is a learnable gate bias. Setting $R = 1$ and $\beta_1 = 0$ recovers the minimal reader.
Value / output update	$u_t = \alpha_t \cdot W_v e_t$ $o_t = u_t + \text{ShortConv}(u_t)$ The Engram update is integrated inside the native block.	Shared-value multi-branch aggregation: $o_t = \frac{1}{R} \sum_{r=1}^R \alpha_t^{(r)} \cdot v_t$ with $R = 4$ in our best setting, followed by residual injection: $h_t \leftarrow h_t + o_t$ No ShortConv is used.
Branch structure	Multi-branch behavior comes from the native backbone design (e.g., mHC / hyper-connection branches).	Branch count R controls reader-side flexibility: one shared W_v , but branch-specific $W_k^{(r)}$, normalization, and gate bias.
Injection location	Engram is a native within-block component, applied before later sublayers in the transformer block.	Reader is attached via forward hooks and injected after selected layer outputs. The implementation supports one or more injection layers; the best OpenQA artifact in this paper uses dual-layer injection at layers 2 and 10 (0-based).
Training regime	Joint native model training with Engram as part of the backbone architecture.	Two-stage transfer: (1) train source memory + source reader, (2) freeze memory and fit a target-side reader for transfer.

Table 7: Formula-level comparison between original Engram and our transfer-oriented reader. We replace the native within-block module with a target-side reader over frozen hashed memory.

513 C Implementation Details

514 **Models.** We use pretrained models from three families—Pythia [3], TinyLlama [29], and
515 QWen3.5 [22]—to span different architectures, tokenizers, and scales. The main source memories
516 are trained with Pythia-160M ($d=768$, GPT-NeoX tokenizer), QWen3.5-0.8B ($d=1536$, QWen tok-
517 enizer), and QWen3.5-9B ($d=3584$, QWen tokenizer). The main targets are Pythia-410M ($d=1024$),
518 QWen3.5-{2B, 4B, 9B}, and TinyLlama-1.1B ($d=2048$, LLaMA tokenizer). For peer-sized transfer,
519 we additionally use Phi-4-mini-instruct ($d=3072$, Phi tokenizer) as both source and target.

520 **Memory configuration.** The source Engram memory uses $N_{\text{max}} = 3$ (bigram and trigram), $K = 4$
521 hash heads per order, table size $M = 65,536$, and head dimension $d_{\text{head}} = 64$, yielding $d_{\text{mem}} = 512$
522 and approximately 33.5M total memory parameters.

Dataset. All perplexity transfer experiments use WikiText-103 [17] with sequence length 512. Training uses the standard train split; evaluation uses the test split with 34 or 36 batches depending on the target tokenizer.

Training protocol. Unless otherwise stated, we train the source Engram memory end-to-end for 50M tokens in Phase 1, then freeze the exported tables, attach them to the target model, and train only the target-side reader for 20M tokens in Phase 2.

Reader regimes. The main 3×3 transfer matrix and the ablations use the minimal single-layer, single-branch reader from §3.2. The main OpenQA study uses the stronger dual-layer 4-branch reader. Unless explicitly noted, the appendix-supported downstream and alignment studies use a deeper but still single-branch reader, typically injected at layer 10 with one branch; we use those results to probe the boundary conditions of transfer rather than to compare directly against the dual-layer 4-branch OpenQA system.

Conditions and metrics. Across transfer experiments, we compare a *Baseline* target with no memory, a *Transferred* condition with a trained frozen source memory, and a *Random memory* condition with an untrained table of the same architecture. Unless noted otherwise, all conditions use three seeds (42, 137, 2024). We report test perplexity (PPL, $\downarrow$) with 95% bootstrap confidence intervals and use paired *t*-tests on per-batch perplexities. All experiments run on a single NVIDIA H100 NVL GPU.

D Supplementary Evidence for RQ1

These results extend the transferability claim in RQ1 beyond the main 3×3 matrix.

D.1 Full Cross-Architecture Matrix

Appendix Table 8 provides the exact cell values behind the main-text heatmap, including the corresponding random-memory controls.

Table 8: Full cross-architecture transfer matrix: test PPL ($\downarrow$) for 3 source memories $\times$ 3 target models. Mean $\pm$ std over 3 seeds. Deltas shown next to the transferred result are relative to the no-memory baseline; negative values indicate better PPL.

Target	Baseline	Transferred	Random
<i>Pythia-160M source</i>			
Pythia-410M	21.90	21.56 $\pm$ 0.05 -1.6%	21.86 $\pm$ 0.00
QWen3.5-4B	10.81	10.08 $\pm$ 0.00 -6.8%	10.33 $\pm$ 0.00
TinyLlama-1.1B	10.63	9.50 $\pm$ 0.00 -10.6%	10.00 $\pm$ 0.02
<i>QWen3.5-0.8B source</i>			
Pythia-410M	22.94	21.38 $\pm$ 0.01 -6.8%	22.12 $\pm$ 0.02
QWen3.5-4B	10.52	9.59 $\pm$ 0.00 -8.9%	10.23 $\pm$ 0.00
TinyLlama-1.1B	10.78	9.10 $\pm$ 0.04 -15.7%	9.34 $\pm$ 0.02
<i>QWen3.5-9B source</i>			
Pythia-410M	22.60	21.64 $\pm$ 0.01 -4.3%	22.29 $\pm$ 0.02
QWen3.5-4B	10.53	9.44 $\pm$ 0.02 -10.3%	10.23 $\pm$ 0.01
TinyLlama-1.1B	10.35	9.09 $\pm$ 0.01 -12.2%	9.31 $\pm$ 0.01

D.2 Same-Tokenizer Transfer

Table 9 isolates the easiest transfer setting, where source and target share the Pythia family and tokenizer, and asks whether frozen memory still helps once we compare against parameter-matched alternatives.

The result is still favorable to transferred memory. It outperforms both random memory and the iso-parameter LoRA baselines, while kNN-LM remains a stronger but much more expensive non-parametric reference because it requires a retrieval datastore and nearest-neighbor search at inference time.

Table 9: Same-tokenizer transfer (Pythia-160M $\rightarrow$ Pythia-410M). Mean $\pm$ std over 3 seeds. Deltas in the PPL column are relative to the no-memory baseline; negative values indicate better PPL. LoRA and Cross-LoRA are iso-parameter baselines (1.03M parameters, same 20M token budget). KNN-LM uses a 5M-token datastore with $k=1024$ retrieval at inference time.

Condition	Test PPL $\downarrow$	95% CI	Params / Cost
Baseline (no memory)	21.90	[20.59, 23.36]	—
<i>Transfer Conditions</i>			
Random memory	21.86 $\pm$ 0.00 -0.2%	[20.55, 23.31]	1.05M
Transferred memory	21.56 $\pm$ 0.05 -1.6%	[20.23, 22.97]	1.05M
<i>Parameter-Matched Baselines</i>			
LoRA (rank 7)	23.32 $\pm$ 0.78 +6.5%	[21.93, 24.89]	1.03M
Cross-LoRA [26]	23.93 $\pm$ 0.60 +9.3%	[22.53, 25.50]	1.03M
<i>External Retrieval Reference</i>			
KNN-LM [13] [†]	20.35 -7.1%	[19.30, 21.48]	5M-tok store

[†] Non-parametric; requires $k=1024$ nearest-neighbor search per token at inference.

553 D.3 Cross-Tokenizer Transfer

554 Table 10 moves to the harder Pythia $\rightarrow$ TinyLlama case, where tokenizer mismatch is unavoidable
555 and transfer depends on the shared canonicalization pipeline.

Table 10: Cross-tokenizer transfer (Pythia-160M $\rightarrow$ TinyLlama-1.1B). Mean $\pm$ std over 3 seeds. Deltas in the PPL column are relative to the no-memory baseline; negative values indicate better PPL. Reader parameters: 2.10M.

Condition	Test PPL $\downarrow$	95% CI	Params
Baseline (no memory)	10.63	[9.96, 11.33]	—
<i>Transfer Conditions</i>			
Random memory	10.00 $\pm$ 0.02 -5.9%	[9.36, 10.68]	2.10M
Transferred memory	9.50 $\pm$ 0.00 -10.6%	[8.87, 10.17]	2.10M

556 Even in this setting, transferred memory reduces PPL from 10.63 to 9.50 and clearly beats random
557 memory. This supports the claim that the exported memory is not tied to the source tokenizer as long
558 as the target reader is given a compatible canonical interface.

559 D.4 Peer-to-Peer Transfer

560 Table 11 tests whether transfer remains useful once source and target are both strong mid-scale
561 models rather than a small source feeding a much larger target.

Table 11: Peer-to-peer transfer between Phi-4-mini (3.8B) and QWen3.5-4B. Source memories trained on WikiText-103 (50M tokens). Mean $\pm$ std over 3 seeds. Deltas shown next to the transferred result are relative to the no-memory baseline; negative values indicate better PPL.

Direction	Baseline	Transferred	Random
Phi-4-mini $\rightarrow$ QWen3.5-4B	10.53	9.46 $\pm$ 0.00 -10.1%	10.23 $\pm$ 0.01
QWen3.5-4B $\rightarrow$ Phi-4-mini	12.61	11.46 $\pm$ 0.00 -9.2%	12.33 $\pm$ 0.01

562 The gains remain bidirectional: Phi $\rightarrow$ QWen improves by 10.1%, and QWen $\rightarrow$ Phi improves by
563 9.2%. Random memory is substantially weaker in both directions, so the effect is not explained by
564 simply attaching another module to the target backbone.

565 D.5 Target Model Scaling

566 Table 12 keeps the source fixed and scales the target from 2B to 9B parameters to check whether
567 transfer disappears once the target becomes stronger.

Table 12: Target model scaling: QWen3.5-0.8B source $\rightarrow$ QWen3.5-{2B, 4B, 9B}. Mean $\pm$ std over 2–3 seeds. Deltas shown next to the transferred result are relative to the no-memory baseline; negative values indicate better PPL. Baselines are eval-only (no reader training).

Target	Baseline	Transferred	Random
QWen3.5-2B	13.67	11.74 $\pm$ 0.01 -14.1%	12.62 $\pm$ 0.01
QWen3.5-4B	10.52	9.59 $\pm$ 0.00 -8.9%	10.23 $\pm$ 0.00
QWen3.5-9B	9.27	8.51 $\pm$ 0.00 -8.3%	9.00 $\pm$ 0.00

The gains persist across all three target sizes. The largest relative improvement appears at 2B, but even the 9B target still benefits, which suggests that stronger backbones reduce but do not eliminate the value of the transferred memory.

D.6 CKA Analysis of Backbone Representations

This figure provides a representation-level view of why transfer need not track backbone similarity monotonically.

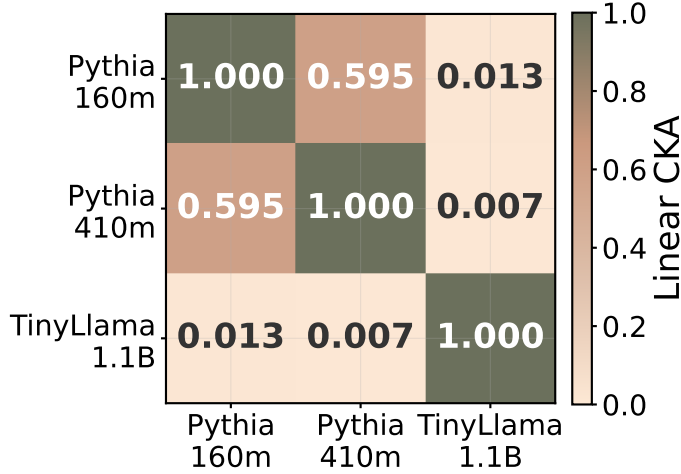


Figure 3: Linear CKA similarity between mean-pooled backbone representations at layer $\lfloor L/3 \rfloor$ for the three models shown in the matrix.

The key qualitative pattern is that backbone similarity is only a weak proxy for transferability. In particular, the paper’s strongest cross-tokenizer case involves Pythia and TinyLlama, whose intermediate representations are much less aligned than same-family pairs, reinforcing the main text’s claim that extractability through the target reader matters more than raw representational similarity.

E Supplementary Evidence for RQ2

These results support the claim that transfer is data-efficient on the target side.

E.1 Scaling Efficiency

Table 13 compares target-side data efficiency directly by tracking the same transfer setting across 5M, 20M, and 50M training tokens.

Transferred memory is already strong after 5M tokens and essentially saturates by 20M, whereas the from-scratch memory only catches up near 50M despite updating far more parameters. This is the cleanest appendix evidence that Phase 2 is reusing structure from the source memory rather than relearning it from the target corpus alone. The 50M transferred run is reported at its best early-stopped checkpoint, which is why it matches rather than improves over the 20M result.

Table 13: Scaling analysis: test PPL ($\downarrow$) as a function of training tokens for Pythia-160M $\rightarrow$ Pythia-410M. Mean over 2 seeds. Early stopping (patience 3) is applied to the transferred condition at 50M tokens, and the reported value is the best checkpoint reached before divergence.

Condition	Params	5M tok	20M tok	50M tok
Transferred	1.05M	21.75	21.52	21.52[†]
From scratch	34.6M	21.89	21.79	21.55

[†]Early stopped at step 1,500 ($\approx$ 12M tokens); val PPL diverges beyond this point.

F Supplementary Evidence for RQ4

These tables characterize when transferred memory helps beyond perplexity and where its main limitations appear.

F.1 Downstream Task Evaluation

The main-paper downstream table (Table 4) already reports the full knowledge-intensive evaluation under FineWeb-Edu-aligned Phase 2 training, so we do not repeat the same table here.

The improvements are selective rather than universal. RTE and SciQ benefit most consistently, BoolQ improves mainly for the 2B and 9B targets, and TruthfulQA stays flat to slightly negative across all scales. A plausible explanation is that TruthfulQA penalizes over-reliance on popular but false web associations, whereas Engram-style memory is trained to make corpus-level associations easier to inject. This is why the main paper frames transferred memory as most useful when paired with an aligned reader and evaluation target, rather than as a generic accuracy booster.

F.2 Domain-Aligned Source Memory

Table 14 compares two source-corpus families under matched Phase 2 adaptation: a broad educational web corpus (FW) and a narrower QA-heavy corpus (Nemo HQ-DQA).

Table 14: Domain alignment ablation with corpus-matched Phase 2 readers. Positive gains are shown in green and negative changes in red.

Target	Source	RTE $\uparrow$	BoolQ $\uparrow$	OpenBookQA $\uparrow$	SciQ $\uparrow$	TruthfulQA $\uparrow$	RACE $\uparrow$
QWen-0.8B	FW-avg	+6.1	-0.6	+0.7	+1.8	-0.1	-0.2
	Nemo-avg	+7.9	+4.1*	+1.1	+8.4*	-0.8	+1.1*
QWen-2B	FW-avg	+2.7	+3.2	+0.3	+3.5	-0.4	-0.0
	Nemo-avg	+8.7*	+14.6*	+0.5	-0.7	+0.0	+1.1*
QWen-4B	FW-avg	+1.3	-0.2	+0.6	+0.9	-0.5	+0.1
	Nemo-avg	+0.4	+0.6	+0.2	-6.3	-2.9	-0.4
QWen-9B	FW-avg	+0.5	+0.6	+0.2	+2.7	-0.7	+1.0
	Nemo-avg	-4.6	+2.1*	-0.3	-3.7	+0.2	+1.4

*Nemo-avg > FW-avg by ≥ 2 percentage points.

The main pattern is that domain-specialized memory helps most on smaller targets and QA-style tasks. Nemo-avg is especially strong on QWen-0.8B and QWen-2B for BoolQ, RTE, and SciQ, but the advantage is less uniform on 4B and 9B targets, showing that specialization helps most when the target still needs strong external support.

Table 15 controls for corpus size directly on QWen-2B by comparing an 8B specialist corpus against a 26B generic one.

Here the smaller specialist source wins decisively on BoolQ and RTE and also improves RACE, while the larger generic web corpus produces much weaker gains. This isolates corpus structure, not raw token count, as the dominant factor in downstream transfer quality.

F.3 Reader Alignment

Table 16 probes the failure mode suggested in the main text: stronger source memories can look worse downstream if the target-side reader is trained on a mismatched Phase 2 objective.

Table 15: Corpus structure vs. size on QWen-2B: accuracy change ($\Delta\%$) under corpus-matched Phase 2 readers. Positive gains are shown in **green** and negative changes in **red**.

Source corpus	BoolQ $\uparrow$	RTE $\uparrow$	OBQA $\uparrow$	SciQ $\uparrow$	TQA $\uparrow$	RACE $\uparrow$
HQ-DQA (8B, STEM Q&A)	+15.3 $\pm$ 0.3	+8.9 $\pm$ 4.7	+0.3 $\pm$ 0.2	+0.9 $\pm$ 3.0	+0.3 $\pm$ 0.2	+1.4 $\pm$ 0.4
HQ (26B, organic web)	+3.0 $\pm$ 0.4	+1.2 $\pm$ 2.1	+0.5 $\pm$ 0.1	-0.5 $\pm$ 0.0	+0.2 $\pm$ 0.3	-0.4 $\pm$ 0.2
FW-avg (reference)	+3.2	+2.7	+0.3	+3.5	-0.4	-0.0

Table 16: Reader alignment: downstream accuracy change ($\Delta\%$) under mismatched vs. matched Phase 2 adaptation. Positive gains are shown in **green** and negative changes in **red**.

Source	Phase 2	BoolQ $\Delta\% \uparrow$	RTE $\Delta\% \uparrow$	SciQ $\Delta\% \uparrow$
<i>Mismatched Phase 2 (WikiText)</i>				
10M	WikiText	+8.3 $\pm$ 0.5	+7.7 $\pm$ 3.9	-4.5 $\pm$ 0.5
50M	WikiText	+6.6 $\pm$ 0.4	+6.1 $\pm$ 4.7	-1.8 $\pm$ 0.6
200M	WikiText	+0.0 $\pm$ 0.0	+2.9 $\pm$ 4.7	-1.4 $\pm$ 0.6
<i>Matched Phase 2 (HQ-DQA)</i>				
10M	HQ-DQA	+14.1 $\pm$ 1.2	+10.7 $\pm$ 4.0	-2.3 $\pm$ 3.0
50M	HQ-DQA	+15.3 $\pm$ 0.3	+8.9 $\pm$ 6.2	+0.9 $\pm$ 3.7
200M	HQ-DQA	+14.0 $\pm$ 0.8	+6.6 $\pm$ 4.9	+2.9 $\pm$ 4.6

Under WikiText Phase 2 adaptation, increasing source training from 10M to 200M tokens steadily hurts BoolQ transfer. Once Phase 2 is moved to the matched HQ-DQA distribution, however, all three source budgets recover to roughly 14–15% BoolQ gain, which shows that the apparent degradation is primarily an alignment problem at the reader rather than a problem with the frozen memory itself.

Table 17 gives a mechanistic view of the same effect through gate statistics.

Table 17: Gate activation statistics during downstream BoolQ evaluation versus Phase 2 WikiText training. Longer source training suppresses the gate globally; BoolQ shows a markedly lower mean gate, while the near-closed-gate fraction rises on both distributions under mismatch.

Source tokens	Gate mean		Frac. < 0.1	
	WikiText	BoolQ	WikiText	BoolQ
10M	0.614	0.539	5.6%	3.6%
50M	0.609	0.486	9.1%	2.9%
200M	0.455	0.352	17.5%	15.3%

Longer source training suppresses the gate more strongly overall, and this is visible on BoolQ prompts both in the lower mean gate and in the rise of near-closed gates. At 200M source tokens, the mean gate on BoolQ drops to 0.352 and the fraction of near-closed gates rises to 15.3%, while WikiText remains slightly higher at 17.5% on the same threshold metric, consistent with broad suppression under reader mismatch rather than a BoolQ-only effect.

F.4 Mixed-Corpus Memory

Table 18 tests whether a strong specialist memory can be broadened without destroying the downstream gains that make it useful in the first place.

The answer is mostly yes. A 50/50 HQ-DQA + FineWeb-Edu mix preserves the DQA aggregate and slightly exceeds the specialist reference, while sequential mixing and the orthogonal code mixture also retain most of the specialist advantage. The main tradeoff is that broader mixtures help coverage more than they help the QA-focused aggregate itself.

F.5 Out-of-Domain Generalization

Table 19 checks whether the transferred memory systematically distorts language modeling behavior on unrelated corpora.

Table 18: Mixed-corpus memory study on QWen3.5-2B. Positive gains are shown in **green** and negative changes in **red**. The two rightmost columns report aggregate scores: DQA-agg = mean(BoolQ, RTE, SciQ); Br-agg = mean(OBQA, TruthfulQA, RACE).

Design	BoolQ $\uparrow$	RTE $\uparrow$	OBQA $\uparrow$	SciQ $\uparrow$	TQA $\uparrow$	RACE $\uparrow$	DQA-agg $\uparrow$	Br-agg $\uparrow$
<i>Single-corpus references</i>								
HQ-DQA ref.	+15.3 $\pm$ 0.3	+8.9 $\pm$ 4.7	+0.3 $\pm$ 0.2	+0.9 $\pm$ 3.0	+0.3 $\pm$ 0.2	+1.4 $\pm$ 0.4	+8.37	+0.65
FW-Edu ref.	+2.5 $\pm$ 1.7	+1.9 $\pm$ 0.2	-0.0 $\pm$ 0.2	+3.2 $\pm$ 0.6	-0.2 $\pm$ 0.2	+0.1 $\pm$ 0.1	+2.54	-0.06
<i>Mixed or broadened memories</i>								
50/50 mix	+14.3 $\pm$ 1.1	+11.7 $\pm$ 3.5	+0.4 $\pm$ 0.4	-0.5 $\pm$ 3.8	+0.3 $\pm$ 0.3	+0.5 $\pm$ 0.3	+8.50	+0.40
Sequential HQ $\rightarrow$ FW	+13.6 $\pm$ 1.0	+11.9 $\pm$ 2.8	-0.0 $\pm$ 0.2	-1.7 $\pm$ 2.7	+0.1 $\pm$ 0.4	+0.2 $\pm$ 0.3	+7.93	+0.11
Orthogonal + Code	+14.4 $\pm$ 0.9	+9.8 $\pm$ 2.6	+0.4 $\pm$ 0.3	-0.4 $\pm$ 3.4	-0.2 $\pm$ 0.7	+1.3 $\pm$ 0.4	+7.94	+0.50

Table 19: Out-of-domain evaluation: test PPL ($\downarrow$) for baseline vs. transferred memory. Negative Δ indicates improvement.

Target Model	Dataset	Baseline	Transferred	Δ (%)
Pythia-410M	LAMBADA	41.00	41.08	+0.2%
	WikiText-2	22.73	22.39	-1.5%
	C4	25.33	25.26	-0.3%
TinyLlama-1.1B	LAMBADA	23.69	23.76	+0.3%
	WikiText-2	10.70	10.53	-1.5%
	C4	11.82	11.89	+0.6%

The OOD effect is mostly neutral. Both targets get a modest improvement on WikiText-2, while LAMBADA and C4 stay very close to the baseline in either direction. This supports the paper’s boundary claim that transferred memory is not universally helpful, but it is also usually not catastrophically harmful outside the aligned regime.

G Complexity

The reader adds little theoretical inference overhead. For Pythia-410M ($d_{\text{mem}} = 512$, $d_B = 1024$), the extra work is about 1M multiply-adds per token, compared with about 4M for a single attention layer. At $d_B = 4096$, the reader requires about 4M multiply-adds per token versus about 67M for one attention layer.

H Ethics and Broader Impact

Transferred memory raises both opportunities and risks. On the positive side, a frozen external memory can reduce repeated backbone retraining and make factual updates more modular. On the negative side, the same mechanism can propagate memorized or provenance-sensitive content across multiple target models if the source memory is not audited. Engram’s advantage is that its stored knowledge is explicit and addressable: entries can be inspected, removed, or replaced without retraining the backbone. We discuss privacy and extraction-boundary considerations further in §A.6.

I Limitations

Scale and availability. Our smallest source model, Pythia-160M, is far below frontier LLM scale. The cross-architecture results show that frozen-memory extraction is already visible in this regime, but the stronger test is transfer from publicly available multi-billion-parameter Engram models, which do not yet exist.

Tokenizer diversity. Our cross-tokenizer experiments cover GPT-NeoX, LLaMA, QWen, and Phi tokenizers, but they do not cover byte-level or Unigram-heavy settings. More aggressive tokenizer mismatch may require a stronger key pipeline than the word-boundary canonicalization used here.

Reader scope. Our main transfer matrix uses a minimal linear reader, and our OpenQA study adds only one stronger family of readers: shared-value multi-branch and multi-layer injection. More distant model pairs may require genuinely nonlinear readers, different injection placements, or joint optimization of addressing and reading. The gate is also a boundary case: it is crucial when corpus mismatch suppresses useful memory, yet it is not the main source of gains in the cleanest in-domain settings.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Introduction explicitly frame the paper as a study of frozen-memory extraction across backbones and identify reader quality as a key determinant of the gains. These claims match the transfer matrix, OpenQA reader ablations, and the stated scope limitations.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The paper contains a dedicated Limitations section covering model scale, tokenizer coverage, and reader scope, and the Conclusion explicitly narrows the claims to Engram-style hashed memory and aligned factual/reasoning-heavy settings.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present formal theorems or proof-based theoretical results; it provides problem formulation, equations, and empirical analysis only.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 3–4 and the appendix specify the models, datasets, memory configuration, optimization protocol, token budgets, seeds, ablations, and evaluation settings needed to reproduce the reported transfer and OpenQA results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

- (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The submission provides an anonymized public repository URL in the abstract for code and reproducible setups.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The Experimental Settings section and appendix specify the datasets, sequence lengths, model families, memory size, source and target training stages, learning rates, schedules, seeds, evaluation metrics, and benchmark suites used in the paper.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The paper reports mean $\pm$ std across seeds, 95% bootstrap confidence intervals for perplexity, and paired t -tests where appropriate, and it states these conventions in the Experimental Settings section.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The paper reports the GPU type used for the experiments, but it does not provide per-run wall-clock time, memory footprint, or total project compute for all experimental groups.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: To the best of our knowledge, the research complies with the NeurIPS Code of Ethics; the paper preserves anonymity and discusses provenance, privacy, and responsible reuse concerns in the Ethics Considerations section.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Ethics Considerations section discusses positive impacts such as modular memory updates and reduced retraining cost, and negative impacts such as provenance-sensitive or memorized content being propagated across target models.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This submission does not release new model checkpoints, memory tables, or datasets, so no release-specific safeguard policy is described here.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: The paper cites the original datasets and model families used in the experiments, but it does not explicitly enumerate the licenses and terms of use for each external asset in the manuscript.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The submission does not claim a new public release of datasets or checkpoints beyond the anonymized code and reproduction repository already linked in the paper.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

972 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
973 or other labor should be paid at least the minimum wage in the country of the data
974 collector.

975 **15. Institutional review board (IRB) approvals or equivalent for research with human**
976 **subjects**

977 Question: Does the paper describe potential risks incurred by study participants, whether
978 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
979 approvals (or an equivalent approval/review based on the requirements of your country or
980 institution) were obtained?

981 Answer: [N/A]

982 Justification: The paper does not involve human subjects research.

983 Guidelines:

984 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
985 with human subjects.

986 • Depending on the country in which research is conducted, IRB approval (or equivalent)
987 may be required for any human subjects research. If you obtained IRB approval, you
988 should clearly state this in the paper.

989 • We recognize that the procedures for this may vary significantly between institutions
990 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
991 guidelines for their institution.

992 • For initial submissions, do not include any information that would break anonymity (if
993 applicable), such as the institution conducting the review.

994 **16. Declaration of LLM usage**

995 Question: Does the paper describe the usage of LLMs if it is an important, original, or
996 non-standard component of the core methods in this research? Note that if the LLM is used
997 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
998 scientific rigor, or originality of the research, declaration is not required.

999 Answer: [N/A]

1000 Justification: The core method and experiments do not use LLMs as a non-standard method-
1001 ological component beyond the pretrained backbones being studied.

1002 Guidelines:

1003 • The answer [N/A] means that the core method development in this research does not
1004 involve LLMs as any important, original, or non-standard components.

1005 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1006 be described.